

Plano de Testes

EasyGis

Avaliação P2 — Engenharia de Software, 2026.1

Equipe EasyGis

Versão 1.0 — 09 de maio de 2026

Sumário

1	Objetivo	3
2	Escopo	3
2.1	No escopo	3
2.2	Fora do escopo	3
3	Estratégia de Teste	3
3.1	Níveis	3
3.2	Abordagem	4
3.3	Critérios de aceitação	4
4	Ambiente de Teste	4
5	Casos de Teste	4
5.1	RF-05 — Cálculo de NDVI	5
5.2	RF-06 — Cálculo de EVI	5
5.3	RF-10 — Estatísticas Descritivas	5
5.4	RF-11 — Histograma	6
5.5	Casos de Borda (CT-05)	6
5.6	RF-20 — Endpoint /health	6
5.7	Endpoint Raiz, Produtos e Validação	6
5.8	Conversão de Coordenadas (CT-10)	7
6	Matriz de Rastreabilidade Requisito × Caso de Teste	7
7	Riscos e Mitigações	7
8	Critério de Saída	8

1. Objetivo

Definir a estratégia, escopo e casos de teste para validar o MVP do sistema, com foco nos requisitos funcionais críticos de processamento de imagens Sentinel-2 (NDVI, EVI, estatísticas) e nos endpoints HTTP da API.

2. Escopo

2.1 No escopo

- **Cálculo de índices espectrais** (NDVI, EVI) — funções puras do `SentinelProcessor`
- **Estatísticas e histograma** — agregações sobre arrays NumPy
- **Endpoints HTTP** — `/`, `/health`, `/products`, validação de entrada de `/calculate-index`
- **Conversão de coordenadas** — geração de polígonos Shapely a partir de coordenadas geográficas
- **Tratamento de casos de borda** — divisão por zero, NaN, arrays vazios

2.2 Fora do escopo

- **Integração com produto Sentinel-2 real** — exigiria download de aproximadamente 1 GB de dados; testado manualmente na demonstração.
- **Testes de UI** (frontend Next.js) — fora do escopo da entrega P2 acordado com o professor; testes manuais via demonstração.
- **Performance e carga** — sistema é local, single-user; não há requisito de carga.
- **Endpoints `/api/experiments/run` e `/api/classification/classify`** — exigem produto Sentinel-2 real para teste end-to-end. Validação por inspeção manual.

3. Estratégia de Teste

3.1 Níveis

Nível	Ferramenta	Cobertura Pretendida
Unitário	pytest	Funções puras de <code>sentinel_processor.py</code> (cálculo de índices, estatísticas)
Integração	pytest + <code>TestClient</code>	Endpoints HTTP, validação Pydantic
Manual	Demonstração ao vivo	Fluxo completo end-to-end com produto Sentinel-2 e KML real

3.2 Abordagem

- **Caixa branca** nas funções de cálculo: validação contra resultados matemáticos conhecidos.
- **Caixa preta** nos endpoints: requisições HTTP com payloads válidos/inválidos, verificação de status code e estrutura de resposta.
- **Fixtures sintéticas** (NumPy) substituem o I/O de imagens reais, permitindo execução rápida (< 1 s) e reproduzível.

3.3 Critérios de aceitação

- 100% dos casos de teste planejados executados.
- Taxa de sucesso $\geq 95\%$.
- Falhas analisadas e documentadas no Relatório de Testes (09-relatorio-de-testes.pdf).

4. Ambiente de Teste

Item	Especificação
Sistema operacional	macOS 15 (Darwin 25.4.0)
Python	3.12.13
pytest	9.0.3
Gerenciador de pacotes	uv 0.11.3
Dependências	pyproject.toml grupo [dependency-groups.dev]

Comando de execução:

```
uv sync --group dev
uv run pytest tests/ -v --cov=api --cov-report=term-missing
```

5. Casos de Teste

Convenção de identificação

- **CT-XX.Y** → Caso de teste número XX, sub-caso Y
- Vinculado ao requisito funcional correspondente em 03-especificacao-requisitos-SRS.md

5.1 RF-05 — Cálculo de NDVI

ID	Caso	Entrada	Resultado Esperado
CT-01.1	NDVI no intervalo válido	Bandas NIR e Red sintéticas (uint16, 100×100)	Array com $\min \geq -1$ e $\max \leq 1$
CT-01.2	Vegetação saudável produz NDVI alto	NIR alto (3000–8000), Red baixo (500–2000)	$\text{mean} > 0,4$
CT-01.3	Fórmula matemática correta	NIR = 8000, Red = 2000	NDVI = 0,6 (= 6000/10000) com tolerância 10^{-6}
CT-01.4	Tratamento de divisão por zero	NIR = 0, Red = 0	Todos os pixels NaN (não crash)
CT-01.5	NDVI zero quando NIR == Red	NIR = Red = 3000	NDVI $\approx 0,0$

5.2 RF-06 — Cálculo de EVI

ID	Caso	Entrada	Resultado Esperado
CT-02.1	EVI no intervalo válido	NIR, Red, Blue sintéticos	$\min \geq -1$ e $\max \leq 1$ (clipping aplicado)
CT-02.2	Coefficientes padrão	NIR=8000, Red=2000, Blue=1000	EVI clipado em 1,0
CT-02.3	Coefficientes customizados produzem resultado diferente	Mesmas bandas, $G = 2,0$ vs $G = 2,5$	Resultados numericamente diferentes

5.3 RF-10 — Estatísticas Descritivas

ID	Caso	Entrada	Resultado Esperado
CT-03.1	Todas as chaves presentes	Array NDVI 50×50	Dict contém <code>min</code> , <code>max</code> , <code>mean</code> , <code>median</code> , <code>std</code> , <code>count</code>
CT-03.2	Valores matematicamente corretos	<code>[[0.1, 0.2, 0.3], [0.4, 0.1, 0.2]]</code>	<code>min=0,1; max=0,6; mean=0,35; count=6</code>
CT-03.3	NaN ignorados no cálculo	Array 20×20 com 100 NaN	<code>count = 300 (= 400 - 100)</code>
CT-03.4	Array totalmente NaN	Array 10×10 todo NaN	<code>count = 0; mean = 0,0</code> (sem crash)

5.4 RF-11 — Histograma

ID	Caso	Entrada	Resultado Esperado
CT-04.1	Padrão 50 bins	Array NDVI saudável	<code>len(counts) == 50</code> e <code>len(bins) == 50</code>
CT-04.2	Bins customizados	<code>bins=10</code>	<code>len(counts) == 10</code>
CT-04.3	Soma de counts = total de pixels	Array 50×50	<code>sum(counts) == 2500</code>
CT-04.4	Array vazio	Array 5×5 todo NaN	Listas vazias retornadas (sem crash)

5.5 Casos de Borda (CT-05)

ID	Caso	Entrada	Resultado Esperado
CT-05.1	Conversão uint16 → float32	NIR e Red como uint16	<code>ndvi.dtype == np.float32</code>
CT-05.2	NDVI negativo para água	NIR=500, Red=2000 (água)	Todos os pixels < 0

5.6 RF-20 — Endpoint /health

ID	Caso	Requisição	Resultado Esperado
CT-06.1	Health responde 200	GET /health	HTTP 200
CT-06.2	Estrutura da resposta	GET /health	JSON com <code>status: "healthy"</code> e <code>products_dir: bool</code>

5.7 Endpoint Raiz, Produtos e Validação

ID	Caso	Requisição	Resultado Esperado
CT-07.1	Raiz retorna info da API	GET /	JSON com <code>message</code> , <code>version</code> , <code>endpoints</code>
CT-08.1	Endpoint /products responde 200	GET /products	HTTP 200
CT-08.2	Estrutura JSON correta	GET /products	<code>products</code> (lista) e <code>count</code> (int)
CT-09.1	Campos obrigatórios faltando	POST /calculate-index body vazio	HTTP 422
CT-09.2	Tipo de coordenada inválido	longitude="invalid" (string)	HTTP 422
CT-09.3	<code>index_type</code> desconhecido	<code>index_type="INVALID_I</code>	HTTP 400/404/500

5.8 Conversão de Coordenadas (CT-10)

ID	Caso	Entrada	Resultado Esperado
CT-10.1	Polígono Shapely válido	4 coordenadas formando triângulo fechado	<code>polygon.is_valid == True</code> , <code>area > 0</code>
CT-10.2	Centroide dentro do bbox	Coordenadas em Picos-PI	$-42,6194 \leq c_x \leq -42,6180$ e $-4,8809 \leq c_y \leq -4,8798$

6. Matriz de Rastreabilidade Requisito $\times$ Caso de Teste

Requisito (SRS)	Casos de Teste
RF-04 (Listar produtos)	CT-08.1, CT-08.2
RF-05 (Calcular NDVI)	CT-01.1 a CT-01.5
RF-06 (Calcular EVI)	CT-02.1, CT-02.2, CT-02.3
RF-10 (Estatísticas)	CT-03.1 a CT-03.4
RF-11 (Histograma)	CT-04.1 a CT-04.4
RF-20 (Health check)	CT-06.1, CT-06.2
RNF-04 (Confiabilidade — erros claros)	CT-01.4, CT-03.4, CT-04.4, CT-09.1, CT-09.2, CT-09.3
RNF-05 (Manutenibilidade — separação de camadas)	Toda a suite (testa <code>SentinelProcessor</code> isoladamente das rotas HTTP)

7. Riscos e Mitigações

Risco	Mitigação
Testes dependem de produtos Sentinel-2 reais (~1 GB)	Uso de fixtures sintéticas via NumPy. Apenas testes manuais usam dados reais (na demo).
Ambiente de CI sem GDAL/rasterio	Execução em máquina local com GDAL instalado via Homebrew. Não há CI configurado neste MVP (escopo P2).
Cobertura limitada de funções de I/O (<code>read_band</code> , <code>crop_to_geometry</code>)	Aceitável para o MVP — essas funções são thin wrappers sobre rasterio, validados em demonstração manual.

8. Critério de Saída

A entrega da P2 está pronta quando:

- ✓ Todos os 28 casos de teste foram executados
- ✓ Taxa de sucesso $\geq 95\%$ (**atingido: 100%**)
- ✓ Relatório consolidado com evidências está disponível em `09-relatorio-de-testes.pdf`
- ✓ Suite executa em < 30 s (**atingido: ~ 17 s na primeira execução; < 1 s em *re-runs***)